

Table of Contents

Compute Platform Decision Document: Azure Container Apps Jobs for the Offline Content Pipeline

Document type: Technical Decision Record **Scope:** Offline content pipeline (Scraper, Chunk & Index, Content Freshness) **Status:** Draft for governance review

1. Purpose & Problem Statement

1.1 Problem Statement

The Digital Assistance offline content pipeline currently runs on **Azure Container Apps Jobs**. A question has been raised on whether **Azure Functions** should be used instead, since the reference architecture depicts ingestion, chunking, embedding, and indexing as function-shaped components.

This document evaluates that question strictly against the **documented, current capabilities and limits of both platforms**, applied to the **actual execution profile of the three scripts in this pipeline** — not a generic platform comparison. All technical limits cited below are sourced from official Microsoft Learn documentation, referenced inline.

1.2 What This Document Does Not Cover

This document does not evaluate Azure Batch, virtual machines, or Kubernetes — those are addressed separately in the pipeline’s Low-Level Design document. This document is scoped narrowly to the Container Apps Jobs vs. Azure Functions question, as requested.

1.3 Scope of the Workload Being Evaluated

Component	What it does	Execution shape
Scraper	Fetches ~297 approved URLs using a headless browser (craw4ai + Playwright); some pages require programmatic interaction with dropdown UI elements	Long-running, browser-automation-dependent, manual trigger
Chunk & Index	Chunks scraped content, generates augmented questions per chunk via LLM calls, generates embeddings, writes to Azure AI Search	Long-running, LLM-call-bound, manual trigger, full run processes the entire approved content set
Content Freshness	Re-scrapes all approved URLs	Long-running (full re-scrape

Component	What it does	Execution shape
	nightly, hash-compares against the index, and reprocesses only changed pages	every run), automated nightly trigger

All three components currently run as **containerized jobs**, deployed as a single Docker image with Playwright and its system dependencies pre-installed.

2. Executive Summary

Recommendation: Retain Azure Container Apps Jobs for all three offline pipeline components.

The determining factor is not cost or general platform preference — it is that **the Scraper component has a hard, documented dependency (headless browser automation with system-level libraries) that Azure Functions’ serverless hosting tiers do not support**, and the **Chunk & Index and Content Freshness components have execution durations that fall outside Azure Functions’ default and, in the Consumption tier, maximum execution limits.**

Sections 3–5 lay out the specific, sourced constraints. Section 6 addresses where Azure Functions *could* be used, and what that would require.

3. Constraint 1 — Browser Automation Dependency (Scraper Component)

3.1 The Requirement

The Scraper component uses Playwright to render pages and, for a subset of approved URLs, programmatically interact with dropdown UI elements that change page content without a URL change. This requires a full headless browser engine with its underlying operating system library dependencies (e.g., `libnss3`, `libatk-bridge2.0-0`, and related packages).

3.2 What Azure Functions’ Documentation States

- The **Consumption plan** runs in a sandboxed environment with no administrative/root access, meaning system-level libraries required by a full browser engine **cannot be installed**. This is documented directly: “*Azure Functions Flex Consumption plan... operates in a sandboxed environment without root access,*” and Linux system libraries cannot be installed via package managers on this tier.[1]
- The same sandboxing constraint applies in practice on the **Premium plan without a custom container** — Microsoft’s own GitHub issue tracker for Azure Functions documents a Premium-plan failure when attempting to run Playwright: “*Host system is missing a few dependencies to run browsers... npx playwright install-deps*” — while confirming the same code runs on Consumption only because a narrower, unofficial

workaround for headless Chromium exists there, not because system dependencies are actually resolvable.[2]

- Microsoft’s documented workaround for running library-dependent workloads on Azure Functions is: **“Use Premium Plan with a custom container – full control to install libraries.”**[1] In other words, Microsoft’s own guidance for this exact class of problem is to package the workload as a **custom container**, which is the Azure Container Apps Jobs model already in use.

3.3 Implication

Running the Scraper on Azure Functions is only possible by attaching a custom Docker container to a Premium-plan Function App. At that point, the workload is no longer using Azure Functions’ native serverless execution model — it is running a container, with Azure Functions acting as an additional hosting/billing layer on top of infrastructure that Container Apps Jobs already provides natively and more directly.

4. Constraint 2 — Execution Duration (All Three Components)

4.1 Azure Functions Documented Timeout Limits

Per official Microsoft documentation, the following default and maximum execution durations apply per plan:[3]

Plan	Default timeout	Maximum timeout
Consumption plan	5 minutes	10 minutes (hard limit)
Flex Consumption plan	30 minutes	Unbounded, with conditions (see 4.2)
Premium plan	30 minutes	Unbounded, with conditions (see 4.2)
Dedicated (App Service) plan	30 minutes	Unbounded, with conditions (see 4.2)

Additionally, **regardless of plan or configured timeout, any HTTP-triggered function is capped at 230 seconds** to respond, due to the Azure Load Balancer’s fixed idle timeout — this limit cannot be raised through configuration.[3]

4.2 The “Unbounded” Caveat, as Documented

Microsoft’s own documentation qualifies “unbounded” execution on Flex Consumption and Premium plans with explicit conditions, not an unconditional guarantee:

“There is no maximum execution timeout duration enforced. However, the grace period given to a function execution is 60 minutes during scale-in for the Flex Consumption and Premium plans, and a grace period of 10 minutes is given during platform updates.”[3]

In practice this means: platform maintenance events and scale-in behaviour can terminate a long-running function execution with as little as a 10-minute grace period, **even when `functionTimeout` is configured for unbounded duration**. This is a documented operational risk for any execution expected to run for multiple hours unattended, which is the profile of a full offline-pipeline run.

4.3 What This Means for This Project's Components

- **Scraper:** A full run across 297 approved URLs — including per-option dropdown interaction on a subset of pages — is a long-running, sequential browser-automation task. This exceeds the Consumption plan's 10-minute hard ceiling outright, and carries real termination risk on Premium/Flex plans due to the scale-in/platform-update grace-period behaviour documented above.
- **Chunk & Index:** This component makes a language-model call *per chunk* (question generation) across the entire approved content set on a full run. This is I/O-bound, multi-call, and — as with the scraper — a long-running batch operation, not a short event-triggered execution, which is what Azure Functions' documented execution model is designed around.
- **Content Freshness:** Performs a full re-scrape of all approved URLs on every nightly run, before any hash comparison or reprocessing occurs (see the pipeline's Low-Level Design document, Section 6.2, for why a full re-scrape is required rather than a partial check). This inherits the same duration profile as the Scraper component.

4.4 Azure Container Apps Jobs — Documented Execution Model

Official Microsoft documentation describes Container Apps Jobs as purpose-built for exactly this execution shape:

“Jobs are used to start containerized tasks that run for a finite duration and then exit. Jobs are best suited for tasks such as data processing, machine learning, resource cleanup, or any scenario that requires on-demand processing.”[4]

Execution duration for a Container Apps Job is controlled by the `--replica-timeout` parameter, which is **set by the workload owner, not constrained by a platform-imposed ceiling** — official quickstart examples configure this from 30 minutes up to an hour and beyond, depending on the job's actual needs.[5] This directly matches the offline pipeline's requirement: long-running, finite-duration, containerized batch execution, with no dependency on an HTTP request/response cycle or an event-driven trigger shape.

5. Constraint 3 — Trigger Model Fit

5.1 What This Project Needs

- Scraper and Chunk & Index: **manual trigger**, run on demand after an approved-URL-list change.
- Content Freshness: **scheduled trigger**, nightly, unattended.

Both are supported natively by Container Apps Jobs, which documents three trigger types — Manual, Schedule (cron), and Event — with Manual and Schedule triggers matching this project’s needs exactly: “*Manual jobs are triggered on demand... Scheduled jobs are triggered at specific times and can run repeatedly.*”[4]

5.2 Why This Matters Relative to Functions

Azure Functions’ documented strength is event-driven and workflow-oriented background processing — Microsoft’s own comparison guidance states Functions are “particularly well suited for event-driven and workflow-oriented background workloads, where work is initiated by external signals and coordination is a core concern,” while Jobs are recommended “for workloads that are intentionally designed to run as a single execution unit without fan-out, trigger-based scaling, or workflow orchestration... where parallelism, retries, and state handling are implemented directly within the application.”[6]

This project’s pipeline is precisely the second case: each component is a single, self-contained execution unit with its own internal retry and state handling (deterministic content hashing, pre-flight validation before index mutation — see the pipeline’s Low-Level Design document, Sections 5–6). This is Microsoft’s own documented rationale for choosing Jobs over Functions.

6. Where Azure Functions Could Apply — And What It Would Require

This section addresses the ingestion/chunking/embedding/indexing breakdown shown in the reference architecture, evaluated honestly rather than dismissed outright.

6.1 Feasible Sub-Scope

The **Chunk & Index** component’s internal stages — chunking, embedding, indexing — have no browser-automation dependency once scraped content already exists. In isolation, these stages are CPU-bound (chunking) or I/O-bound (embedding, indexing) operations without the Playwright constraint described in Section 3.

6.2 What Would Be Required to Run This on Azure Functions

- A **Premium or Flex Consumption plan** (not Consumption), to obtain the higher default/configurable timeout — Consumption’s 10-minute hard ceiling is not viable for a full-content-set run.[3]
- Given the per-chunk LLM call pattern, a full run’s duration must be measured against the documented 60-minute scale-in grace period risk noted in Section 4.2 — an unattended job that runs materially longer than that window carries a real, Microsoft-documented risk of mid-run termination on Functions, a risk that does not exist on Container Apps Jobs’ owner-configured `--replica-timeout` model.
- Splitting a currently single-process pipeline into multiple chained functions (chunking — embedding — indexing) introduces distributed-system concerns not present today: message-queue orchestration between stages, idempotency and ordering guarantees across asynchronous executions, and — critically — the pipeline’s existing pre-flight validation safety gate (which today works because all processing for a page happens in

one place before any index deletion occurs) would need to be redesigned as a distributed check spanning multiple function executions.

- The Scraper component would remain outside this scope entirely, per Section 3, and would continue to require Container Apps Jobs (or an equivalent custom-container platform) regardless of what is decided for the remaining stages.

6.3 Assessment

This is technically achievable but represents a genuine architecture change with real engineering effort — not a hosting-platform swap. It is not recommended as part of the current scope, given the Scraper’s hard platform constraint already anchors this pipeline to a container-native platform, and splitting only part of the pipeline onto a second platform (Functions) would introduce two operational models to maintain instead of one, without removing the Container Apps dependency.

7. Decision Matrix

Requirement	Azure Functions (Consumption)	Azure Functions (Premium/Flex, custom container)	Azure Container Apps Jobs
Run headless browser with system dependencies	Not supported — sandboxed, no root access[1]	Supported, but requires the same custom-container approach as Container Apps Jobs[1][2]	Supported natively[4]
Execution duration for a full pipeline run	Hard 10-minute ceiling[3]	Configurable, but with documented scale-in/platform-update termination risk beyond 60 minutes[3]	Owner-configured - - replica - timeout, no platform-imposed ceiling[5]
Manual + scheduled trigger support	Supported (with timeout caveats above)	Supported (with timeout caveats above)	Supported natively, both trigger types[4]
Fit per Microsoft’s own platform guidance	—	—	Recommended for “single execution unit... without fan-out... where parallelism, retries, and state handling are implemented directly within the application”[6]
Additional	N/A (not viable as-	High (custom	None — current

Requirement	Azure Functions (Consumption)	Azure Functions (Premium/Flex, custom container)	Azure Container Apps Jobs
orchestration engineering required for this pipeline	is)	container + distributed pre-flight validation redesign)	design already fits the model

8. Recommendation

Retain **Azure Container Apps Jobs** as the execution platform for all three offline pipeline components (Scraper, Chunk & Index, Content Freshness). This is directly supported by official Microsoft documentation on:

- Sandboxing and system-dependency limitations on Azure Functions' serverless tiers (Section 3),
- Documented execution-duration limits and scale-in termination risk on Azure Functions (Section 4),
- Microsoft's own guidance on when to choose Jobs over Functions for this execution shape (Section 5).

A partial migration of the Chunk & Index sub-stages to Azure Functions (Section 6) is technically possible but is not recommended at this time, given the effort required to redesign the pipeline's existing safety guarantees for a distributed execution model, and given that the Scraper component's hard browser-automation constraint means Container Apps Jobs remains a required part of this pipeline's architecture regardless of that decision.

9. References

[1] Microsoft Q&A, "*Can I install linux libraries to Azure Functions which sku is Flex Consumption plan?*" — Microsoft-moderated answer confirming Flex Consumption's sandboxed, no-root environment, and recommending Premium Plan with a custom container as the documented workaround.

<https://learn.microsoft.com/en-us/answers/questions/2283109/can-i-install-linux-libraries-to-azure-functions-w>

[2] Azure/Azure-Functions GitHub repository, Issue #2140, "*[Premium] NodeJS Azure Function works in Serverless/Consumption but not Premium.*" Documents a Playwright browser-launch failure on the Premium plan due to missing host-system dependencies.

<https://github.com/Azure/Azure-Functions/issues/2140>

[3] Microsoft Learn / MicrosoftDocs official reference, "*Function app timeout duration*" — authoritative table of default and maximum execution timeouts by hosting plan, including the documented Flex Consumption / Premium scale-in grace period and platform-update grace period. <https://github.com/MicrosoftDocs/azure-docs/blob/master/includes/functions->

timeout-duration.md (also published at <https://learn.microsoft.com/en-us/azure/azure-functions/functions-scale>)

[4] Microsoft Learn, “*Jobs in Azure Container Apps*.” Official overview of Container Apps Jobs, trigger types, and recommended use cases.

<https://learn.microsoft.com/en-us/azure/container-apps/jobs>

[5] Microsoft Learn, “*Create a Job in Azure Container Apps*” (CLI quickstart). Documents the `-- replica-timeout` parameter and example configurations. <https://learn.microsoft.com/en-us/azure/container-apps/jobs-get-started-cli>

[6] Microsoft Community Hub / Azure App Service Team Blog, “*Rethinking Background Workloads with Azure Functions on Azure Container Apps*.” Official Microsoft guidance comparing when to choose Azure Functions vs. Container Apps Jobs for background workloads. <https://techcommunity.microsoft.com/blog/appsonazureblog/rethinking-background-workloads-with-azure-functions-on-azure-container-apps/4496861>

End of document.